

Basanos

Benchmarking LLM Agent Reliability Detection

A ReliAgent Validation Study · Executive Summary

HDG Forge, an HD Gregory LLC company · 2026

What Was Tested

ReliAgent is a reliability-monitoring API for LLM agent tool calls. It sits alongside an agent's tool-calling loop and screens for eight failure modes — **hallucination**, **sycophantic agreement**, **context degradation**, **repetition loops**, **confidence collapse**, **schema violations**, **parameter drift**, **timeout**, and **response integrity** — returning a structured pass/warn/fail verdict with supporting evidence on every call. Basanos answers a single question: **does ReliAgent actually detect what it claims to detect?**

The Verdict

All eight detectors were validated. Across 1,800 trials, ReliAgent achieved a 0% false positive rate on seven of eight detectors and 100% true positive rate on all eight (with one harness measurement gap on a single threshold mode of the timeout detector, documented below). The study also surfaced three genuine product defects — all identified and corrected before publication.

Results at a Glance

Each detector was tested with 100 clean trials (measuring false positive rate) and 100 violation trials (measuring true positive rate). All figures reported with 95% confidence intervals in the full report.

Detector	True Positive Rate	False Positive Rate	Source
hallucination	100%	0%	ExpertQA (published)
sycophantic_gap_fill	100%	2.3% *	sycophancy-eval (published)
context_degradation	100%	0%	Lost in the Middle + synthetic
repetition_loop	100%	0%	Synthetic hash injection
confidence_collapse	100%	0%	sycophancy-eval (published)
schema_violation	100%	0%	JSONSchemaBench Glaiveai2K (published)
parameter_drift	100%	0%	Synthetic two-call sequences
timeout	100% †	0%	Synthetic latency values
response_integrity	100%	0%	Synthetic dict payloads (5 sub-checks)

** The 2.3% FPR on sycophantic_gap_fill reflects common filler words appearing in non-sycophantic contexts in the real dataset — within the expected range for a phrase-pattern detector on natural language, consistent with the detector's documented behavior.*

† timeout Mode B (adaptive threshold) recorded 51.5% TPR due to a harness design error: violation latencies were set below the detector's 1000ms adaptive floor. The 16 non-fires are correct product behavior, not misses. Modes A (caller-supplied) and C (default 10,000ms) both recorded 100% TPR across 67 combined trials.

How the Study Was Conducted

Two phases, eight detectors total

Basanos-1 (Pilot): 5-10 trials per detector against real published benchmarks using live Claude Sonnet 5 responses. Purpose: confirm each detector's actual trigger mechanism before committing to a full harness. This step caught two significant mismatches between assumed and real behavior before any inference budget was spent on a wrong approach.

Basanos-2 Track A: Scaled the six pilot-validated detectors to N=200 trials each (100 clean + 100 violation) against higher-quality sources with programmatic violation injection for auditable ground truth.

Basanos-2 Track B: Completed the two remaining detectors and response_integrity using purpose-built synthetic harnesses — the correct instrument for structural, non-language-based detectors whose trigger conditions are precisely defined.

Key methodological decisions

- Mechanism-first: every detector's actual trigger was confirmed via direct test calls before any data pipeline was built. This prevented two harness misdesigns that would have spent inference budget on sources that couldn't exercise the real trigger.
 - Provenance-labeled: every trial states exactly where its input came from. Nothing is presented as organic real-world behavior unless it is.
 - Per-caller isolation: history-dependent detectors use Redis state keyed on caller_ip; each trial was assigned a unique simulated IP to prevent cross-trial history contamination.
 - No-overwrite audit trail: every experiment run is retained in the GitHub repository under a numbered directory, including superseded runs, so the full history is permanently inspectable.
-

Product Findings — Three Defects Surfaced and Corrected

A secondary goal of this study was to find real defects in ReliAgent through adversarial testing. Three genuine defects were found:

Finding 1: False pass on unsupported-keyword schemas (corrected)

A schema relying entirely on an unsupported JSON Schema keyword (anyOf, oneOf, \$ref, etc.) previously returned 'pass' even when the response clearly violated it. A census of 9,542 real-world schemas found this affected ~12% of real function-calling schemas. Fixed: these schemas now return a distinct schema_partially_verified signal.

Finding 2: Guaranteed false positives in single-tool environments (corrected)

The repetition_loop detector included a check for the same tool being called consecutively. In single-tool agent environments (e.g. a coding agent where every call is bash_exec), this fired on every call sequence regardless of whether looping was occurring. Fixed: the check was removed. The hash-based check is the correct and sufficient signal.

Finding 3: Annotation keywords spuriously flagged (corrected)

Annotation-only fields (description, title, \$schema, default) inside property sub-schemas were treated as unsupported validation keywords, triggering spurious partial-verification warnings on virtually every real-world schema with documented properties. Fixed: annotation keywords are now filtered at every nesting level.

All three findings were corrected in the live product before this report was published. The fact that adversarial benchmark testing surfaced these defects — and that they were all fixable — validates the approach: rigorous testing is a productive ongoing practice for this product category, not a one-time certification exercise.

What This Study Does and Does Not Claim

What is validated: ReliAgent's detectors fire on their documented trigger conditions and stay quiet on clean inputs. At N=100 per direction with 95% confidence intervals, this is a statistically powered result.

What is not claimed: cross-model generalization (all language-based trials used Claude Sonnet 5); organic firing rates in real production agent traffic (structural detectors were tested with synthetic trigger injection, which measures behavior on the trigger condition in isolation); or exhaustive JSON Schema coverage (ReliAgent's 11-keyword supported subset covers 88% of real function-calling schemas, with the remainder returning an honest partial-verification signal).

Audit trail: all 1,800 trial-level records, harness code, and methodology documentation are publicly available at github.com/HDGForge-Labs/basanos.

Reports: github.com/HDGForge-Labs/basanos/tree/main/reports

ReliAgent is available at reliagent.net · HDG Forge, an HD Gregory LLC company · 2026